GPU Passthrough & vGPU

AI/ML & Graphics Workloads on KVM

GPU-accelerated workloads are the fastest-growing segment of virtualization. KVM supports full GPU passthrough (dedicated GPU per VM), NVIDIA vGPU (shared GPU across VMs), and Intel SR-IOV GPU. This guide covers IOMMU setup, vfio-pci binding, CUDA validation, and migrating GPU VMs from VMware to KVM.

NVIDIA vGPU | PCI Passthrough | IOMMU | CUDA | Inference VMs | KubeVirt GPU | AMD ROCm

GPU Virtualization Options

Three approaches to GPU access in VMs — choose based on workload and sharing needs.

GPU Virtualization Comparison

Method	Sharing	Performance	Live Migration	License Cost	GPU Support	Best For
PCI Passthrough	1 GPU : 1 VM	100% native	No	$0	Any PCIe GPU	Training, HPC, rendering
NVIDIA vGPU	1 GPU : 2-32 VMs	85-95%	Yes	$$ (licensed)	NVIDIA Tesla/Ampere/Hopper	VDI, shared inference
Intel SR-IOV GPU	1 GPU : 4-8 VMs	80-90%	No	$0	Intel Flex/Arc	Media transcode, light ML
Mediated Device (mdev)	1 GPU : N VMs	70-90%	Partial	Varies	NVIDIA, Intel	Framework-level sharing
virtio-gpu (virgl)	Shared	30-60%	Yes	$0	Host GPU	Desktop VMs, basic 3D

VMware vs KVM GPU Support

VMware GPU Limitations

- vSphere GPU passthrough: DirectPath I/O (manual, per-host)
- NVIDIA vGPU on VMware: requires vSphere Enterprise Plus
- VMware SVGA 3D: limited OpenGL 4.1, no CUDA
- No AMD GPU passthrough support (officially)
- Broadcom licensing: GPU hosts cost same as CPU-only
- vMotion with GPU: only NVIDIA vGPU (not passthrough)
- Cost: $6,175/CPU + NVIDIA vGPU license = $12K+/host

KVM GPU Advantages

- PCI passthrough: any GPU, zero overhead, no extra license
- NVIDIA vGPU: fully supported on KVM (same as VMware)
- AMD ROCm: supported with passthrough (AI/ML on AMD)
- Intel Flex Series: SR-IOV GPU sharing, no license
- Multiple GPUs per VM: NVLINK-aware allocation
- KubeVirt: GPU operator + device plugin = K8s native
- Cost: $0 hypervisor + optional vGPU license

PCI Passthrough Setup

Dedicate a physical GPU to a VM for 100% native performance — ideal for training and HPC.

Step-by-Step GPU Passthrough

```
# Step 1: Enable IOMMU in BIOS + kernel
# Intel: intel_iommu=on iommu=pt
# AMD: amd_iommu=on iommu=pt
grubby --update-kernel=ALL --args="intel_iommu=on iommu=pt"
reboot

# Step 2: Identify GPU PCI address and IOMMU group
lspci -nn | grep -i nvidia
# 41:00.0 3D controller [0302]: NVIDIA Corporation A100 [10de:20b2] (rev a1)
# 41:00.1 Audio device [0403]: NVIDIA Corporation [10de:1aef] (rev a1)

# Verify IOMMU group (all devices in group must be passed through)
find /sys/kernel/iommu_groups/ -type l | grep "41:00"
# /sys/kernel/iommu_groups/56/devices/0000:41:00.0
# /sys/kernel/iommu_groups/56/devices/0000:41:00.1

# Step 3: Bind GPU to vfio-pci driver
echo "10de 20b2" > /sys/bus/pci/drivers/vfio-pci/new_id
echo "10de 1aef" > /sys/bus/pci/drivers/vfio-pci/new_id

# Persistent via modprobe.d
echo "options vfio-pci ids=10de:20b2,10de:1aef" > /etc/modprobe.d/vfio.conf
echo "vfio-pci" > /etc/modules-load.d/vfio.conf

# Step 4: Blacklist NVIDIA host driver (so it doesn't grab the GPU)
echo "blacklist nouveau" > /etc/modprobe.d/blacklist-gpu.conf
echo "blacklist nvidia" >> /etc/modprobe.d/blacklist-gpu.conf
dracut --force
```

Libvirt XML for GPU Passthrough

GPU Device Assignment

```
<hostdev mode='subsystem' type='pci'
         managed='yes'>
  <source>
    <address domain='0x0000' bus='0x41'
             slot='0x00' function='0x0'/>
```

VM Configuration for GPU

```
<!-- Q35 machine type (required for PCIe) -->
<os>
  <type machine='pc-q35-8.2'>hvm</type>
  <loader type='pflash'>
    /usr/share/OVMF/OVMF_CODE.fd
```

```
  </source>
  <address type='pci' domain='0x0000'
           bus='0x06' slot='0x00'
           function='0x0'/>
</hostdev>

<!-- Also pass audio device (same IOMMU group) -->
<hostdev mode='subsystem' type='pci'
         managed='yes'>
  <source>
    <address domain='0x0000' bus='0x41'
             slot='0x00' function='0x1'/>
  </source>
</hostdev>
```

```
  </loader>
</os>

<!-- CPU: host-passthrough for AVX-512 -->
<cpu mode='host-passthrough'/>

<!-- Memory: pin for GPU DMA -->
<memoryBacking>
  <hugepages/>
  <locked/>
</memoryBacking>

<!-- Verify in guest -->
# nvidia-smi
# CUDA test: python3 -c "import torch; print(torch.cuda.is_available())"
```

NVIDIA vGPU on KVM

Share a single GPU across multiple VMs with NVIDIA vGPU — supports live migration.

NVIDIA vGPU Profiles

GPU	Profile	FB Memory	Max VMs	Use Case	Perf vs Bare Metal
A100 (80GB)	A100-1Q	1 GB	32	VDI, light inference	~5%
	A100-5Q	5 GB	16	ML inference, CAD	~15%
	A100-10Q	10 GB	8	Training (small models)	~25%
	A100-20Q	20 GB	4	Training (medium models)	~50%
	A100-40Q	40 GB	2	Large model training	~90%
L40S (48GB)	L40S-4Q	4 GB	12	Inference + rendering	~15%
	L40S-12Q	12 GB	4	LLM inference	~50%
	L40S-24Q	24 GB	2	Fine-tuning, RAG	~85%
H100 (80GB)	H100-20Q	20 GB	4	Inference	~50%
	H100-80Q	80 GB	1	Full GPU (MIG alternative)	~95%

vGPU Setup on KVM

```
# Install NVIDIA vGPU Manager (requires license)
rpm -i nvidia-vgpu-kvm-550.90.05.x86_64.rpm
systemctl enable --now nvidia-vgpud nvidia-vgpu-mgr

# List available vGPU types
mdevctl types
# nvidia-256 (A100-5Q): 5120 MB, max instances: 16
# nvidia-257 (A100-10Q): 10240 MB, max instances: 8

# Create vGPU instance
mdevctl start -u $(uuidgen) -p 0000:41:00.0 --type nvidia-256

# Or persistent:
mdevctl define -u $(uuidgen) -p 0000:41:00.0 --type nvidia-256 -a
```

```
# Libvirt XML for vGPU
# <hostdev mode='subsystem' type='mdev' managed='no' model='vfio-pci' display='on'>
#   <source>
#     <address uuid='a618089c-523f-11ec-bf63-0242ac130002'/>
#   </source>
# </hostdev>
```

AI/ML Workload Migration

Migrating GPU-accelerated VMs from VMware to KVM with CUDA validation.

GPU VM Migration Workflow

Step 1: Export from VMware

- Remove VMware SVGA adapter from VM settings
- Remove DirectPath I/O GPU assignment
- Export VM as OVA (without GPU passthrough)
- GPU driver stays installed in guest OS
- `h2kvmctl export --server vcenter --vm ml-training-01`
- Note: GPU is host-specific, not part of VM image

Step 2: Convert & Deploy

- Convert VMDK to QCOW2 with h2kvmctl
- h2kvmctl adds GPU passthrough to domain XML
- `h2kvmctl convert --source ml-training.vmdk --gpu-passthrough 41:00.0 --emit-domain-xml --deploy-libvirt`
- Auto-detects NVIDIA driver in guest OS
- Sets machine type to Q35, enables UEFI
- Configures hugepages and locked memory

Step 3: Validate CUDA

```
# In guest VM after boot
nvidia-smi
# GPU: NVIDIA A100 80GB
# Driver: 550.90.05
# CUDA: 12.4

# Test CUDA
python3 -c "
import torch
print(f'CUDA: {torch.cuda.is_available()}')
print(f'GPU: {torch.cuda.get_device_name(0)}')
print(f'Memory: {torch.cuda.get_device_properties(0).total_mem / 1e9:.1f} GB')
t = torch.randn(10000, 10000, device='cuda')
print(f'Matrix mul: OK')
"
```

Step 4: Performance Validation

- Run original training benchmark on KVM
- Expected: 98-100% of bare metal performance
- Check: NCCL multi-GPU communication (if multi-GPU)
- Verify: NVLink bandwidth with `nvidia-smi nvlink -s`
- Test: mixed precision training (TF32, FP16, BF16)
- Compare: time-to-train on VMware vs KVM (should match)

Common Issue: NVIDIA Driver Version

The NVIDIA driver in the guest must match the host vGPU Manager version (for vGPU) or be compatible with the GPU (for passthrough). If the VM was running an older driver on VMware, update it after migration: `dnf install nvidia-driver-550` or download from NVIDIA's website. For passthrough, any compatible driver works. For vGPU, host and guest driver versions must match exactly.

KubeVirt GPU Integration

Schedule GPU workloads on Kubernetes with KubeVirt and NVIDIA GPU Operator.

NVIDIA GPU Operator for KubeVirt

```
# Install NVIDIA GPU Operator (auto-installs drivers, device plugin, etc.)
helm install gpu-operator nvidia/gpu-operator \
  --set driver.enabled=true \
  --set toolkit.enabled=true \
  --set devicePlugin.enabled=true \
  --set sandboxDevicePlugin.enabled=true  # For KubeVirt vGPU

# Verify GPUs discovered
kubectl get nodes -o json | jq '.items[].status.allocatable["nvidia.com/gpu"]'
# "4"  (4 GPUs available)

# KubeVirt VM with GPU passthrough
kubectl apply -f - <<'EOF'
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: ml-training-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          gpus:
            - name: gpu1
              deviceName: nvidia.com/gpu
        resources:
          requests:
            memory: 32Gi
            cpu: "8"
      volumes:
        - name: rootdisk
          persistentVolumeClaim:
            claimName: ml-training-disk
EOF

# Verify GPU in VM
virtctl console ml-training-vm
# nvidia-smi shows GPU inside VM
```

vGPU on KubeVirt

vGPU with Mediated Devices

```
# KubeVirt VM with vGPU
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: inference-vm
spec:
  template:
    spec:
      domain:
        devices:
          gpus:
            - name: vgpu1
              deviceName: nvidia.com/NVIDIA_A100-5Q
        resources:
          requests:
            memory: 16Gi
            cpu: "4"
```

Multi-GPU Training VM

```
# 4x GPU for distributed training
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: llm-training
spec:
  template:
    spec:
      domain:
        devices:
          gpus:
            - name: gpu1
              deviceName: nvidia.com/gpu
            - name: gpu2
              deviceName: nvidia.com/gpu
            - name: gpu3
              deviceName: nvidia.com/gpu
            - name: gpu4
              deviceName: nvidia.com/gpu
        resources:
          requests:
            memory: 256Gi
            cpu: "64"
```

AMD & Intel GPU Support

NVIDIA isn't the only option — AMD ROCm and Intel Flex Series work on KVM.

GPU Vendor Comparison for KVM

Feature	NVIDIA	AMD	Intel	Notes
Passthrough	**Full support**	**Full support**	**Full support**	All vendors work with VFIO
vGPU (shared)	**vGPU (licensed)**	**No vGPU**	**SR-IOV (free)**	Intel SR-IOV = no license
AI/ML framework	**CUDA (dominant)**	**ROCm (growing)**	OneAPI (limited)	CUDA: widest framework support
Inference	**TensorRT**	**MIGraphX**	**OpenVINO**	All competitive for inference
VDI/Desktop	**GRID**	**MxGPU (limited)**	**Flex Series**	NVIDIA GRID most mature
Media transcode	**NVENC**	**VCE/VCN**	**QSV (best $/stream)**	Intel: 30+ streams per Flex 170
Live migration	**vGPU only**	**No**	**No**	Passthrough: no migration for any
Typical card cost	$7K-$30K	$4K-$15K	$2K-$5K	Intel: best value for media/VDI

AMD ROCm on KVM

AMD Instinct MI300X Passthrough

```
# Same VFIO passthrough process as NVIDIA
echo "1002 740f" > /sys/bus/pci/drivers/vfio-pci/new_id

# In guest: install ROCm
amdgpu-install --usecase=rocm

# Verify
rocm-smi
# GPU: AMD Instinct MI300X (192GB HBM3)

# PyTorch with ROCm
```

Intel Flex Series SR-IOV

```
# Enable SR-IOV on Intel Flex 170
echo 7 > /sys/class/drm/card0/device/sriov_numvfs

# Each VF gets ~7GB VRAM (out of 48GB total)
# Libvirt XML:
# <hostdev mode='subsystem' type='pci'>
#   <source>
#     <address bus='0x4d' slot='0x00'
#              function='0x1'/> <!-- VF -->
#   </source>
# </hostdev>
```

```
pip install torch --index-url https://download.pytorch.org/whl/rocm6.0
python3 -c "import torch; print(torch.cuda.is_available())"
# True (ROCm uses CUDA compatibility layer)
```

```
# In guest: media transcode
ffmpeg -hwaccel qsv -i input.mp4 \
  -c:v hevc_qsv output.mp4
# 30+ 1080p streams per Flex 170
```

GPU Workload Architectures

Common deployment patterns for GPU-accelerated VMs on KVM.

Pattern 1: ML Training Cluster

- **Hardware:** 4 servers, 8x A100 80GB each
- **Config:** PCI passthrough, 8 GPUs per VM
- **Network:** InfiniBand or RoCE for NCCL
- **Storage:** NVMe local or Ceph for datasets
- **Use case:** LLM training, 30B+ parameter models
- **Performance:** 100% of bare metal (passthrough)
- **VMware cost:** $300K/yr (licensing + GPU support)
- **KVM cost:** $0 (hypervisor free, GPU same cost)

Pattern 2: Inference Farm

- **Hardware:** 8 servers, 4x L40S each
- **Config:** vGPU, 4 VMs per GPU (48 VMs total)
- **Profile:** L40S-12Q (12GB per VM)
- **Use case:** LLM inference (Llama 70B, Mistral)
- **Concurrent models:** 48 instances
- **VMware cost:** $200K/yr + $96K vGPU licenses
- **KVM cost:** $96K (vGPU licenses only)
- **Savings:** $200K/yr (hypervisor cost eliminated)

Pattern 3: VDI with GPU

- **Hardware:** 6 servers, 2x L40S each
- **Config:** vGPU, 16 users per GPU (192 desktops)
- **Profile:** L40S-3Q (3GB per user)
- **Use case:** CAD/CAM, GIS, engineering
- **Protocols:** SPICE (open), NICE DCV, Parsec
- **VMware Horizon:** $200/user/yr + ESXi
- **KVM + SPICE:** $0 + vGPU license
- **Savings:** $150K/yr (192 users)

Pattern 4: Media Transcode

- **Hardware:** 3 servers, 4x Intel Flex 170 each
- **Config:** SR-IOV, 7 VMs per GPU (84 VMs)
- **Use case:** Live streaming, VOD transcode
- **Capacity:** 30 streams/GPU x 12 = 360 streams
- **Codec:** H.264, H.265, AV1 hardware encode
- **Intel SR-IOV:** No license required (free)
- **VMware cost:** $120K/yr + no SR-IOV support
- **KVM cost:** $0 (Intel SR-IOV free on KVM)

Cost Analysis & Migration Summary

GPU workloads benefit the most from VMware exit — zero hypervisor tax on expensive GPU hardware.

Total Cost: GPU Infrastructure (3-Year, 32 GPUs)

Component	VMware	KVM	Notes
GPU hardware (32x A100 80GB)	$480,000	$480,000	Same hardware cost
Server hardware (8 hosts)	$240,000	$240,000	Same servers
vSphere Enterprise Plus (3yr)	$296,400	**$0**	$6,175/CPU x 16 CPUs x 3yr
vGPU licenses (3yr)	$288,000	$288,000	Same NVIDIA vGPU cost (if used)
vCenter (3yr)	$33,675	**$0**	Not needed for KVM
VMware GPU support add-on	$48,000	**$0**	VMware charges extra for GPU hosts
hyper2kvm migration	—	$0	Apache-2.0, free
Migration labor	—	$30,000	One-time (32 GPU VMs)
3-Year Total	**$1,386,075**	**$1,038,000**	**Savings: $348,075 (25%)**

Without vGPU (passthrough only): VMware $1,098,075 vs KVM $750,000 — savings: **$348,075 (32%)**

GPU + KVM = Maximum Performance at Minimum Cost

PCI passthrough on KVM delivers 100% native GPU performance — identical to bare metal, identical to VMware passthrough, but without $378K in hypervisor licensing over 3 years. For AI/ML workloads where GPU hardware is the major cost, eliminating the hypervisor tax is the single most impactful optimization. hyper2kvm migrates GPU VMs with auto-configured passthrough, UEFI, hugepages, and CUDA validation.